

VPS provisioning and hardening

Follow-along: take a server as it is delivered and close what should not be open

Overview

Definition: Virtual private server

A machine rented from a hosting provider: a share of somebody else's hardware, given its own operating system, its own public address, and a root password mailed to whoever ordered it. What arrives is a running Linux system reachable from the whole internet within a minute of being created, configured for the provider's convenience rather than for the tenant's safety. Everything past that first login is the tenant's work, and nothing does it for them.

Nothing in this lab attacks anything. What it hands you is a **virtual private server** in the state a provider delivers one: root reachable over SSH with a password from any address that can route to it, three services listening on every address the machine holds, and no packet filter at all. You close each of those. By the end the public web service still serves anyone who asks for it, SSH accepts one key from one subnet, the administrative page is reachable over no network at all, and every other port discards what arrives rather than replying to it.

The work is five changes and they are not interchangeable. An account and a key decide who may log in; two `sshd_config` keywords decide which credentials the server will consider; the address a service binds decides which of the machine's addresses it exists on; and a packet filter decides which packets reach a service at all. Three of those five are settings on the service, one is a property of a socket, and one is a rule in the kernel. A learner who reaches the right end state without being able to say which of the five did which has learned a checklist rather than a machine.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab    # first time
cd ~/MiniLab && git pull                                       # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one category per row. Select **Host and service management**, then **VPS provisioning and hardening**, to reach this lab's actions:

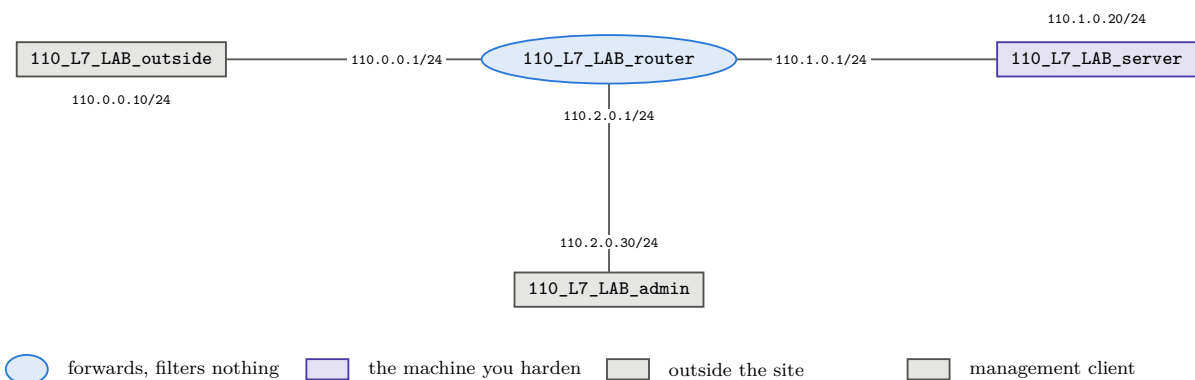
- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its image; later spawns take seconds.
- **Status** prints the lab's state and its success oracle, so you read the before and after without shelling in. It also shows which ports the server answers on, separately for the outside host

and for the administrative client. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.

- **Shell command for a node** lists every node (server, admin, outside, router) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does.
- **Reset to baseline** throws away everything you configured on the server and returns it to the state it was delivered in, without tearing the lab down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup



Four containers on three subnets, with one router between them. The server is the only machine you configure. The administrative client stands in for your own workstation on a management network, and the outside host stands in for everyone else on the internet; you run every port sweep from it, and you never configure it.

Every container is named `110_L7_LAB_` followed by the name in the first column, so the server is `110_L7_LAB_server`.

Device	Interface	Address	Subnet
outside	110-ext	110.0.0.10	external 110.0.0.0/24
router	ext	110.0.0.1	external
	srv	110.1.0.1	server 110.1.0.0/24
	mgmt	110.2.0.1	management 110.2.0.0/24
server	110-srv	110.1.0.20	server
admin	110-mgmt	110.2.0.30	management

The router forwards between all three subnets and filters nothing. That is worth saying plainly, because it decides what the word “management” can mean in this lab: the outside host has a route to `110.2.0.0/24` and the management client has a route to `110.1.0.20`, so putting a machine on a subnet called management gives it no protection whatever. Whatever restricts SSH to that subnet has to be written on the server, and Part 5 is where you write it.

Definition: Listening socket, and the address it is bound to

A service that accepts connections asks the kernel for a socket and names an address and a port to attach it to. The address is not decoration: it decides which of the machine's addresses the service exists on. 0.0.0.0 is the wildcard, meaning every IPv4 address the machine holds now or later; 127.0.0.1 is the loopback address, which only processes on this machine can send to. The binding is fixed when the socket is opened, so changing it means restarting the service, not signalling it. `netstat -tln` lists every listening TCP socket as `<address>:<port>`.

The server runs three services, and each one is a **listening socket** you will look at more than once:

- **sshd on tcp/22**, the OpenSSH server. It is how the machine is administered, and it is delivered accepting a root password from anywhere.
- **lighttpd on tcp/80**, the public site. This is what the server exists to do, and it has to keep serving the outside host when you are finished.
- **A second lighttpd instance on tcp/8080**, an administrative status page. It carries no login of its own and it is delivered on every address the machine holds.

Two things about the containers are fixed and cannot be changed from inside them, because Docker mounts `/proc/sys` read-only there. The router is started with `net.ipv4.ip_forward=1`, which is what makes it forward packets addressed to somebody else. The server is started with `CAP_NET_ADMIN`, which is the privilege `nft` needs to load a ruleset; without it Part 5's first command would be refused.

The tooling is the standard Linux set, and five commands do almost everything:

- **nmap**: send a TCP probe to each of a list of ports and report what came back.
- **netstat -tln**: list this machine's listening TCP sockets and the address each is bound to.
- **ssh, ssh-keygen**: open a session, and generate a key pair.
- **nft**: read and load the kernel's packet filter ruleset.
- **curl**: fetch one URL and print what came back.

Open a shell wherever a step says so with:

```
# any of the four containers: this is how every step below opens a shell
docker exec -it <container> bash
```

Assessment. This lab is graded on nineteen questions, marked inline below as **Question N**, each placed at the step whose output it asks about. Answer all nineteen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step, or from a prediction you make before running the next command.

Part 1: what the server responds to as delivered

Before changing anything, find out what the machine offers to someone who has its address and nothing else. That is one command, and it is the same command an attacker runs first.

Definition: Port sweep, and the three states it reports

Sending one TCP probe to each of a list of ports on one address, and recording what came back for each. The reply decides the state, and there are three. A SYN met with a SYN-ACK means a service accepted the connection, and **nmap** reports **open**. A SYN met with a reset means the machine's TCP stack received it and no service was listening on that port, and **nmap** reports **closed**. A SYN that draws nothing at all means the probe or its reply was discarded somewhere, and **nmap** reports **filtered**. The three are not three properties of a port: they are two different replies plus the absence of either.

1. **Run a port sweep against the server from outside the site.** Four ports: the three the server is running something on, and one that nothing is listening on, which is there so you can see what the third state looks like before anything is filtering.

```
# 110_L7_LAB_outside: the host outside the site
docker exec -it 110_L7_LAB_outside bash
nmap -Pn -n -p 22,80,443,8080 110.1.0.20
```

-n skips name resolution, which nothing in this lab provides. **-Pn** skips **nmap**'s own host-discovery step, which sends probes of its own and abandons a target that replies to none of them. Every sweep in this lab uses it, and Part 5 is where the reason becomes visible.

Question 1. Record the state **nmap** reported for each of the four ports. For the one it reported **closed**, state what the server's TCP stack sent back to produce that reading, and say what **nmap** would have reported instead if the server had sent nothing.

2. **Look at the same three services from the inside.** A sweep says which ports respond; the server's own socket table says which address each service attached itself to.

```
# 110_L7_LAB_server: the machine you are hardening
docker exec -it 110_L7_LAB_server bash
netstat -tln
```

All three read 0.0.0.0 in the local address column, which is the wildcard: each service took every IPv4 address the machine holds. This server holds two, 110.1.0.20 and the loopback address 127.0.0.1, so a connection to any of the three ports can be made to either of them. Part 4 is about the one service for which that is the wrong place to be.

3. **Log in as root, over the network, with a password.** The password the provider mailed over is Trondheim-5182.

```
# 110_L7_LAB_outside: the host outside the site
ssh root@110.1.0.20
```

ssh asks you to confirm the server's host key the first time; type **yes**. Once you are in, look at what you have, then leave with **exit**:

```
# 110_L7_LAB_server: in the session you just opened
id
```

4. **Read what the server recorded.** `sshd` was started with `-e`, which sends its authentication log to standard error, and the lab redirects that to a file:

```
# 110_L7_LAB_server: one command, run from your own terminal
docker exec 110_L7_LAB_server tail -5 /var/log/vps-sshd.log
```

5. **Fetch the administrative page from outside the site.** It carries no login of its own.

```
# 110_L7_LAB_outside
curl http://110.1.0.20:8080/
```

Question 2. Two ports on this server are reachable from outside and should not be: `tcp/22` and `tcp/8080`. One asks for a credential and one does not. Say which of the two you would close first, and justify the order by what an attacker gets from each in the first minute of finding them.

Part 2: an administrative account that is not root

Working as root over SSH puts one account, whose name every attacker already knows, in front of the one service the machine must keep reachable. The replacement is an ordinary account that logs in with a key and becomes root only for the commands that need it. Part 2 builds that account. It closes nothing by itself, and the checks at the end of this part are there to make that concrete: after Part 2 the password still works and root still logs in. Part 3 is what stops them, and it can only be done safely once you have another way in.

Definition: Public-key authentication

A pair of files generated together: a private key, which stays on the client and never crosses the network, and a public key, which is copied to every server the key should open. The client proves it holds the private key by signing a value derived from the session; the server checks that signature against the public key it has on file for the account, in `~/.ssh/authorized_keys`. Nothing reusable crosses the wire, so a server that accepts only keys has nothing an eavesdropper or a compromised server can replay.

1. **Generate the key pair** **public-key authentication** needs, on the administrative client. `-t ed25519` picks the signature algorithm; `-N ''` gives the private key no passphrase, which is what lets the rest of this lab run without a prompt on every command.

```
# 110_L7_LAB_admin: the administrative client
docker exec -it 110_L7_LAB_admin bash
mkdir -p /root/.ssh && chmod 700 /root/.ssh
ssh-keygen -t ed25519 -N '' -f /root/.ssh/id_ed25519
cat /root/.ssh/id_ed25519.pub
```

Question 3. You are about to copy `id_ed25519.pub` to the server, over a session authenticated with the root password an attacker may also hold. Say what someone who reads that file, in transit or on the server afterwards, can do with its contents, and justify the answer from which half of the pair proves possession.

2. **Create the account on the server, and put it in the wheel group.**

Definition: sudo, and the wheel group

`sudo` runs one command as another user, normally root, after checking the caller against the policy in `/etc/sudoers` and asking for the caller's own password. A line in that file whose user field begins with `%` names a system group rather than an account, so `%wheel ALL=(ALL:ALL) ALL` grants the privilege to whichever accounts are members of `wheel`, and group membership becomes the thing that is administered. `wheel` means nothing to `sudo` by itself; it is an ordinary group that Unix systems conventionally use for this, and Alpine ships the line present and commented out.

```
# 110_L7_LAB_server: run from your own terminal, as root
docker exec -it 110_L7_LAB_server bash
adduser -D -s /bin/sh opsadmin
passwd opsadmin                # set it to Bergen-7734
addgroup opsadmin wheel
```

Then uncomment the `wheel` line in `/etc/sudoers`. It is the line reading `# %wheel ALL=(ALL:ALL) ALL`:

```
# 110_L7_LAB_server
sed -i 's/^# %wheel ALL=(ALL:ALL) ALL/%wheel ALL=(ALL:ALL) ALL/' /etc/sudoers
grep '%wheel' /etc/sudoers      # confirm exactly one uncommented line
```

3. **Install the public key.** Paste the line you printed in step 1 in place of `<paste the public key here>`. It is one line however it wraps on your screen.

```
# 110_L7_LAB_server
mkdir -p /home/opsadmin/.ssh
echo '<paste the public key here>' > /home/opsadmin/.ssh/authorized_keys
chown -R opsadmin:opsadmin /home/opsadmin/.ssh
chmod 700 /home/opsadmin/.ssh
chmod 600 /home/opsadmin/.ssh/authorized_keys
```

The two `chmod` lines are not tidiness. `sshd` refuses to read an `authorized_keys` file when that file, the `.ssh` directory, or the home directory above it can be written by anyone other than its owner, because a file another account can write is a file another account can add its own key to. What it enforces is the write bit for group and for other; the read and execute bits are not part of the check. When the check fails, `sshd` declines to read the file and the login falls through to the next method, so from the client's side an ignored key and a missing key look identical.

4. **Read the mode back.**

```
# 110_L7_LAB_server
stat -c '%a %U' /home/opsadmin/.ssh /home/opsadmin/.ssh/authorized_keys
```

Question 4. One of the two modes `stat` prints has four digits rather than three, and you typed a three-digit mode for both. Give both modes as printed, name the extra bit, and say whether its presence changes the outcome of the check described above. Justify the answer from what that check tests.

5. Log in with the key, and use sudo.

```
# 110_L7_LAB_admin: the administrative client
ssh -i /root/.ssh/id_ed25519 opsadmin@110.1.0.20
```

```
# 110_L7_LAB_server: in the session you just opened
id                                # opsadmin, and a member of wheel
sudo nft list ruleset             # asks for Bergen-7734; prints nothing,
    because nothing is loaded
exit
```

6. Check that the old ways in still work. From the administrative client, log in as opsadmin with the password instead of the key:

```
# 110_L7_LAB_admin
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no
opsadmin@110.1.0.20
```

Question 5. The password login you just made succeeded, and so did the key login before it. State what installing the key changed about who can reach this account, and what it did not change.

Question 6. Predict, before Part 3 turns password authentication off in `sshd`, whether `sudo` will still ask you for `Bergen-7734` afterwards and still accept it. Name the component that checks the password in each of the two cases, and use the difference to justify your prediction.

Part 3: sshd stops accepting root, and stops accepting passwords

The account and the key from Part 2 are what make this part safe: there is now a way into the machine that does not depend on either of the two things you are about to switch off. Do not do this part before Part 2 works, on this server or any other.

Definition: Reloading a daemon with SIGHUP

A signal is a number the kernel delivers to a running process, and SIGHUP is the one long-standing convention reserves for “re-read your configuration”. `sshd` reads `/etc/ssh/sshd_config` when it starts and again when it receives SIGHUP, and at no other time. Between an edit and a signal, the file on disk and the running daemon disagree, and nothing on either side says so. `kill -HUP <pid>` sends it, and `sshd` records its own process id in `/run/sshd.pid`.

1. Read what the daemon is running now. `sshd -T` prints the configuration after every default and every include has been resolved, in lower case, which is not the same as reading the file: a keyword may appear twice, commented, or not at all.

```
# 110_L7_LAB_server: run from your own terminal, as root
docker exec -it 110_L7_LAB_server bash
sshd -T | grep -E 'permitrootlogin|passwordauthentication|kbdinteractive'
```

2. Turn off root login and password authentication.

Edit the existing lines rather than appending new ones. `sshd` uses the first value it obtains for a keyword, so a setting appended below one that is already uncommented earlier in the file has no effect at all, and `sshd -T` will keep reporting the old value:

```
# 110_L7_LAB_server
sed -i 's/^PasswordAuthentication .*/PasswordAuthentication no/'
    /etc/ssh/sshd_config
sed -i 's/^PermitRootLogin .*/PermitRootLogin no/' /etc/ssh/sshd_config
sshd -T | grep -E 'permitrootlogin|passwordauthentication'
```

Question 7. `sshd -T` now reports both keywords as `no`, and you have sent the daemon no signal. Predict whether a password login from the administrative client succeeds right now. Write the prediction down, then run the next step and record what actually happened.

3. Try a password login before reloading.

```
# 110_L7_LAB_admin: the administrative client
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no
opsadmin@110.1.0.20
```

4. Send the daemon **SIGHUP**, and try again.

The reload is a re-exec: `sshd` runs itself again with the options it was started with, which is why the listening socket survives it and why the pid in `/run/sshd.pid` is the one to signal.

```
# 110_L7_LAB_server
kill -HUP $(cat /run/sshd.pid)
```

```
# 110_L7_LAB_admin: the administrative client
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no
opsadmin@110.1.0.20
```

Question 8. Record the refusal line the client printed. `sshd` names in it the authentication methods it is still willing to attempt. List them, and say for each one whether a password could be typed into it.

5. Close the second password path.

Turning off `PasswordAuthentication` is not the whole of turning off passwords.

Definition: Keyboard-interactive authentication

A separate authentication method in the SSH protocol from `password`. Rather than carrying one credential, the server sends the client a list of prompts and the client returns a list of responses, which is what lets it carry a one-time code or a challenge. On a host whose backend for it is PAM, the prompt it sends is commonly the account's ordinary password, so leaving it enabled leaves a second way to type the same secret. It is governed by `KbdInteractiveAuthentication`, which is a different keyword from `PasswordAuthentication` and has its own default of `yes`.

```
# 110_L7_LAB_server
sed -i 's/^#*KbdInteractiveAuthentication .*/KbdInteractiveAuthentication no/'
/etc/ssh/sshd_config
sshd -T | grep kbdinteractive
kill -HUP $(cat /run/sshd.pid)
```

```
# 110_L7_LAB_admin: the administrative client
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no
opsadmin@110.1.0.20
```

Question 9. Give the refusal line as it reads now, alongside the one you recorded two steps ago. Account for the difference between the two, naming the keyword responsible and what **keyboard-interactive authentication** is separately from password authentication.

6. **Check root from outside, and check yourself from the management client.** A hardening step that locks out the operator is a failure, so both halves are checked every time.

```
# 110_L7_LAB_outside: the host outside the site
ssh root@110.1.0.20
```

```
# 110_L7_LAB_admin: the administrative client
ssh -i /root/.ssh/id_ed25519 opsadmin@110.1.0.20 'id -un'
```

```
# 110_L7_LAB_server: one command, run from your own terminal
docker exec 110_L7_LAB_server tail -4 /var/log/vps-sshd.log
```

Question 10. Root can no longer log in at all, and opsadmin can reach root through sudo on any session it opens. Explain what setting **PermitRootLogin no** buys given that second fact. Two things it changes are visible in the log lines you have collected so far; name at least one of them.

Part 4: the administrative page leaves every address but loop-back

Nothing you have done touches tcp/8080. The administrative page has no login to harden, so the question it raises is a different one: which of the machine's addresses should it exist on at all. It is meant for whoever administers the server, and it belongs on none of the network ones.

Part 4 moves it to loopback, which makes it reachable only by a process already running on the server, and then gives the administrative client a way to reach it that does not put it back on the network: an SSH tunnel, which carries the connection inside a session the client has already authenticated.

1. **Move the socket.** `server.bind` names the single address lighttpd attaches its listening socket to, and its default is every local IPv4 interface. The second instance's configuration is `/etc/lighttpd/admin.conf`:

```
# 110_L7_LAB_server: run from your own terminal, as root
docker exec -it 110_L7_LAB_server bash
grep server.bind /etc/lighttpd/admin.conf
sed -i 's|^server.bind .*|server.bind          = "127.0.0.1"|'
    /etc/lighttpd/admin.conf
```

An address is chosen when a socket is opened and a running process cannot move one, so this instance is stopped and started rather than signalled:

```
# 110_L7_LAB_server
kill $(cat /run/lighttpd-admin.pid)
lighttpd -f /etc/lighttpd/admin.conf
netstat -tln | grep 8080
```

2. Sweep tcp/8080 again from outside, and fetch it again.

```
# 110_L7_LAB_outside: the host outside the site
nmap -Pn -n -p 8080 110.1.0.20
curl --max-time 5 http://110.1.0.20:8080/
```

Question 11. Give the state `nmap` reports for tcp/8080 now, alongside the state you recorded for it in Part 1. Nothing on this server is filtering anything yet. Explain the change in terms of what the server's TCP stack did with the probe in each case.

3. Check it from the server itself.

```
# 110_L7_LAB_server
curl http://127.0.0.1:8080/
curl --max-time 5 http://110.1.0.20:8080/
```

Definition: SSH local port forwarding

`ssh -L <local port>:<host>:<hostport>` opens a listening socket on the client. Each connection made to it is carried over the encrypted session, and the connection to `<host>:<hostport>` is then opened by the *server*, from the server. So the host and port name a destination in the server's own view of the network, and 127.0.0.1 in that position is the server's loopback rather than the client's. `-N` asks for no remote command and `-f` puts the client in the background once the session is up.

4. Reach the page through a tunnel. Open one with **SSH local port forwarding** from the administrative client, fetch through it, then close it:

```
# 110_L7_LAB_admin: the administrative client
ssh -i /root/.ssh/id_ed25519 -f -N -L 18080:127.0.0.1:8080 opsadmin@110.1.0.20
curl http://127.0.0.1:18080/
pkill -x ssh
```

Question 12. The direct fetch of 110.1.0.20:8080 from this same client failed and this one succeeded, and the page has not moved. Name the machine that opens the connection

to 127.0.0.1:8080 in the tunnelled case, and say why that machine is allowed to make it.

5. **Look at what the tunnel is permitted to reach.** `AllowTcpForwarding` is yes on this server, which is OpenSSH's default. Ask for a different destination:

```
# 110_L7_LAB_admin: the administrative client
ssh -i /root/.ssh/id_ed25519 -f -N -L 18081:127.0.0.1:80 opsadmin@110.1.0.20
curl --max-time 5 http://127.0.0.1:18081/
pkill -x ssh
```

That fetch returns the public site. Nothing about `opsadmin` was ever meant to grant access to `tcp/80` beyond the ability to log in, and nothing so far has been written that limits it.

6. **Scope the forwarding to the one destination it needs.** `PermitOpen` lists the `host:port` destinations a forwarded channel may be opened to; anything else is refused per channel, so the session still opens and only the forwarded connection fails.

```
# 110_L7_LAB_server
echo 'PermitOpen 127.0.0.1:8080' >> /etc/ssh/sshd_config
sshd -T | grep permitopen
kill -HUP $(cat /run/sshd.pid)
```

```
# 110_L7_LAB_admin: the administrative client
ssh -i /root/.ssh/id_ed25519 -f -N -L 18081:127.0.0.1:80 opsadmin@110.1.0.20
curl --max-time 5 http://127.0.0.1:18081/
pkill -x ssh
```

```
# 110_L7_LAB_server: one command, run from your own terminal
docker exec 110_L7_LAB_server tail -3 /var/log/vps-sshd.log
```

Question 13. Quote the line the server logged for the refused forward. The server sits on one subnet in this lab; describe the set of destinations an account with this server's default `AllowTcpForwarding` could have opened a channel to, and say what `PermitOpen` has reduced that set to.

Part 5: a packet filter that denies by default

Everything so far changed what the services do with a connection. This part changes which packets reach a service at all, and it is the only one of the five that constrains a port nothing is listening on and a service you have not thought about.

Definition: Default-deny policy

A filter written as a list of what is permitted, with everything else discarded, rather than a list of what is forbidden with everything else allowed. The difference is what happens to traffic nobody considered: under default-deny a service started next week is unreachable until a rule names it, and under default-allow it is exposed the moment it binds. Both designs can be written with identical rules; the policy is what separates them.

Definition: Netfilter hook, base chain, and chain policy

The kernel's packet path passes through a fixed set of hooks. The **input** hook is the one every packet addressed to this machine crosses on its way to a local socket. An **nftables** chain given a hook and a priority is a **base chain**: it is registered against that hook and sees those packets. A chain without a hook sees nothing and is only a jump target. Only a base chain carries a **policy**, which decides what happens to a packet that reached the end of the chain without a rule accepting or dropping it. The values are **accept**, which is the default, and **drop**.

Definition: Connection tracking

The kernel records each connection it has seen and, for every packet, which connection it belongs to and what state that connection is in. **ct state new** is a packet that starts a connection; **established** is one belonging to a connection that has carried traffic in both directions; **related** is one starting a new connection that an existing one accounts for, such as an ICMP error about it. A rule matching on state is how a filter distinguishes a connection somebody outside is starting from a reply to one this machine started.

1. **Predict what a bare default-deny policy does.** You are about to create a **base chain** at the **input** hook with policy **drop** and no rules in it at all. You are working through **docker exec**, which is not a network connection to the server, so nothing you do here can shut you out. Answer for the case you would be in on a rented server, where the only way to reach the machine is the SSH session you are typing in.

Question 14. State what happens to the SSH session you are typing in the moment that chain is created, and what happens to a reply arriving for a connection the server itself opened. Name the packets in each case and say which rule matched them.

2. **Build the policy.** Run these on the server, all of them, before checking anything. That is the habit the previous question is about: on a machine reached only over the network, the chain and the rule that keeps your own session alive have to be loaded together, because between the two commands there is no rule that matches it. Rule order matters, and the first rule matters most.

```
# 110_L7_LAB_server: run from your own terminal, as root
docker exec -it 110_L7_LAB_server bash
nft add table inet filter
nft 'add chain inet filter input { type filter hook input priority filter ;
    policy drop ; }'
nft add rule inet filter input ct state established,related accept
nft add rule inet filter input iif lo accept
nft add rule inet filter input tcp dport 80 accept
nft add rule inet filter input ip saddr 110.2.0.0/24 tcp dport 22 accept
nft list ruleset
```

The four rules are, in order: replies to connections **connection tracking** already holds an entry for, so nothing below is consulted for them; anything arriving on the loopback interface, which is what the tunnel's server end needs; the public site, from any source, because that is what the server exists to serve; and SSH, from the management subnet only.

3. **Sweep from both vantages.**

```
# 110_L7_LAB_outside: the host outside the site
nmap -Pn -n -p 22,80,443,8080 110.1.0.20
```

```
# 110_L7_LAB_admin: the administrative client
nmap -Pn -n -p 22,80,443,8080 110.1.0.20
```

Question 15. Give a table of the four ports against the state each sweep reported, one column per vantage. For every port whose two columns differ, name the rule in your ruleset that produced the difference.

Question 16. Take the row of your table for the one port nothing has ever been listening on. Its state in Part 1 and its state now are not the same. Explain the change in terms of what the server does with the arriving SYN in each case, and say which of the two readings tells an attacker more about the machine.

4. Check that the server still does its job.

```
# 110_L7_LAB_outside: the host outside the site
curl http://110.1.0.20/
ping -n -c 2 -W 2 110.1.0.20
```

```
# 110_L7_LAB_admin: the administrative client
curl http://110.1.0.20/
ssh -i /root/.ssh/id_ed25519 opsadmin@110.1.0.20 'id -un'
ssh -i /root/.ssh/id_ed25519 -f -N -L 18080:127.0.0.1:8080 opsadmin@110.1.0.20
curl http://127.0.0.1:18080/
pkill -x ssh
```

Question 17. Your ruleset names no rule for ICMP, and the `ping` above got no reply. Say which rule discarded the echo request. Then say what `nmap`'s default host-discovery step would have concluded about this server, and use that to explain why every sweep in this lab was run with `-Pn`.

5. Prove the loopback rule is load-bearing. `nft -a` prints each rule's handle, which is how a single rule is deleted.

Question 18. Predict what happens to the tunnelled fetch of the administrative page, and to the `curl http://127.0.0.1:8080/` you ran on the server itself, if the `iif lo accept` rule is deleted. Write the prediction down before running the next block.

```
# 110_L7_LAB_server
nft -a list chain inet filter input          # read the handle on the 'iif lo'
      accept' line
nft delete rule inet filter input handle <the handle you just read>
curl --max-time 5 http://127.0.0.1:8080/
```

Put it back, at the top of the chain rather than the bottom, and check the page returns:

```
# 110_L7_LAB_server
nft insert rule inet filter input index 1 iif lo accept
curl http://127.0.0.1:8080/
```

6. **Look at what the SSH rule does and does not do.** It matches on the source address range 110.2.0.0/24.

Question 19. The router between the three subnets forwards everything and filters nothing, and no device in this lab checks that a packet's source address belongs on the interface it arrived on. State what the source-address rule stops, and state what it would not stop if the outside host set its own source address to one inside 110.2.0.0/24. Then name what would still be in that attacker's way once their packet reached tcp/22, and say which part of Part 2 or Part 3 put it there.

Checks: what “done” means

Run **Status** in the TUI, or `scripts/status.sh`. It reads the server rather than changing it, and it sweeps from both vantages so the two open sets sit next to each other. Done is every check ticked, which is these five things at once:

- The public site serves the outside host and the administrative client.
- tcp/22 responds to the management subnet and to nowhere else.
- tcp/8080 responds to no address on the network, and the tunnel still reaches it.
- A password opens no session, and the administrative client's key still does.
- No packet reaches a port your ruleset does not name.

The last line of the output says **HARDENED** when all of them hold. Anything else prints the unticked checks; the first one is the earliest thing to fix, because every later stage is read through it.

If you want to start a part again from the delivered state, **Reset to baseline** throws away the account, the key, both configuration files and the ruleset, and leaves the lab running.

Safety note

This lab is isolated. The four containers are wired to each other with veth pairs and nothing else; no container holds a route off this host, and the addresses in 110.0.0.0/8 exist only inside the lab. Nothing you run here reaches any machine but the three the lab created, and `nmap` is aimed at one address that only exists while the lab is up.

Tear the lab down

When you have finished, remove it. Pick **Teardown** in the TUI, or run `scripts/teardown.sh` directly. It removes this lab's four containers and nothing else.